

# Adaptive Centripetal Pincer Sort

## Relatório técnico de método de ordenação autoral

**Universidade Federal do Pampa - Campus Alegrete**

**Disciplina:** Análise e Projeto de Algoritmos

**Trabalho:** Trabalho Prático 1 - Metodos de Ordenacao Autorais

**Autores:** Marcus Vinicius Morini Querol Junior e Vinicius Da Silva Goncalves

**Data:** 9 de setembro de 2026

# Resumo

Este relatório apresenta o Adaptive Centripetal Pincer Sort, ou ACPS, uma adaptação estrutural autoral de técnicas de Quick Sort com dois pivôs. O método foi projetado para reconhecer entradas monotonicamente ordenadas, reduzir trabalho em conjuntos com poucas chaves distintas e manter particionamento in-place no núcleo do algoritmo. A proposta combina sondagem de monotonicidade, amostragem fixa de cinco elementos, particionamento tripartite com dois limiares, congelamento de platôs, fallback ternário e corte para Insertion Sort em segmentos pequenos.

A corretude foi verificada por 22 testes automatizados, incluindo 3.280 vetores enumerados exaustivamente sobre um domínio ternário, entradas aleatórias até  $N = 10.000$ , duplicatas intensas, tipos numéricos diferentes, preservação da entrada e verificação do contador de comparações. O estudo experimental realizou 625 execuções medidas em cinco distribuições. Em  $N = 10.000$ , o ACPS concluiu entradas ordenadas com 9.999 comparações e nenhuma movimentação, entradas reversas com 19.998 comparações e 10.000 movimentações, e entradas com cinco chaves distintas com média de 55.573 comparações. Em dados aleatórios, apresentou 156.558 comparações médias, abaixo de Quick Sort e DPES na implementação avaliada, mas com mais movimentações do que Quick Sort.

O melhor caso é  $\Theta(N)$ . Sob a hipótese de permutação aleatória e partições esperadamente balanceadas, o caso médio é  $\Theta(N \log N)$ . O pior caso permanece  $\Theta(N^2)$ , pois a amostragem é determinística. O núcleo particiona in-place e usa  $O(\log N)$  de pilha pela eliminação da chamada recursiva da maior partição; a interface pública, entretanto, usa  $\Theta(N)$  de memória total porque copia a entrada.

## 1 Introdução

Algoritmos de ordenação são um contexto clássico para estudar projeto, invariantes, recorrências e efeitos da distribuição da entrada. O objetivo deste trabalho não é propor superioridade universal sobre métodos consolidados, mas construir e analisar criticamente uma combinação própria de mecanismos conhecidos.

O ACPS parte de quatro observações. Primeiro, vetores já ordenados ou em ordem não crescente podem ser tratados em tempo linear. Segundo, dois limiares permitem separar a entrada em três zonas em uma única varredura. Terceiro, repetições exigem cuidado para que uma partição central não seja processada indefinidamente. Quarto, segmentos pequenos tendem a favorecer um método simples com baixa sobrecarga.

A principal contribuição do projeto é a política que coordena esses mecanismos: uma sondagem global, dois limiares obtidos de uma amostra posicional, um corredor central, detecção explícita de estagnação e fallback por pivô mediano. A proposta é apresentada como adaptação estrutural, com as técnicas de origem identificadas na Seção 3.

## 2 Objetivos

O trabalho busca:

- especificar formalmente o ACPS;
- demonstrar sua corretude por invariantes e indução;
- deduzir limites de tempo e espaço;
- caracterizar estabilidade e operação in-place;
- validar a implementação em casos normais, extremos e adversariais;
- comparar o comportamento com Bubble, Selection, Insertion, Merge, Quick Sort e DPES;
- registrar de forma reproduzível tempo, comparações e movimentações;
- declarar as fontes e o uso de ferramentas de inteligência artificial.

### 3 Origem das técnicas e contribuição autoral

O particionamento com dois pivôs, a divisão em três regiões e a seleção de pivôs por pequenas amostras pertencem a família de Dual-Pivot Quicksort. Estudos de Nebel e Wild e de Aumuller e Dietzfelbinger analisam precisamente o impacto da amostragem e das estatísticas de ordem na quantidade de comparações, trocas e acessos.

O uso de Insertion Sort em partições pequenas, a mediana de uma amostra e o tratamento especial de chaves iguais também são otimizações conhecidas de Quick Sort. O DPES fornecido na disciplina serviu como referência direta para a interface de retorno, para a instrumentação e para a ideia de dois limiares adaptativos.

As decisões específicas do ACPS são:

1. executar uma sondagem global de monotonicidade antes da primeira partição;
2. reverter linearmente uma entrada não crescente;
3. amostrar as posições inicial, um quarto, metade, três quartos e final;
4. usar o segundo e o quarto valores da amostra ordenada como limiares  $p_1$  e  $p_2$ ;
5. particionar com ponteiros esquerdo, corrente e direito;
6. congelar o corredor quando  $p_1 = p_2$ ;
7. detectar quando o corredor central ocupa todo o segmento;
8. nesse caso, realizar partição ternária pela mediana da amostra, em vez de aplicar Insertion Sort ao segmento inteiro;
9. processar recursivamente apenas as partições menores e continuar iterativamente na maior.

Assim, a autoria está na composição, nas políticas de decisão e no tratamento da estagnação. Não se afirma que os componentes isolados sejam inéditos.

### 4 Especificação do ACPS

#### 4.1 Convenção de métricas

Uma comparação é cada avaliação relacional entre duas chaves por  $<$ ,  $>$ ,  $=$  ou  $\neq$ . Uma movimentação é uma escrita de chave na lista de trabalho. Uma troca entre posições diferentes corresponde a duas movimentações. Cópias da entrada e atribuições em variáveis auxiliares não entram na contagem, de modo consistente com as baselines do experimento.

Essa contagem inclui, explicitamente, as comparações realizadas ao ordenar localmente a amostra de cinco elementos em `_sample_pivots`. Como a amostra tem tamanho fixo, seu custo de ordenação interna é  $O(1)$  e não altera os limites assintóticos, mas é contabilizado para manter a fidelidade entre o contador reportado e as avaliações relacionais observadas externamente — propriedade verificada pelo teste 20.

#### 4.2 Fases do algoritmo

**Sondagem de monotonicidade.** Pares adjacentes são examinados enquanto ainda for possível que o vetor seja não decrescente ou não crescente. Se for não decrescente, ele já está ordenado. Se for não crescente, a inversão por pares simétricos produz a ordem crescente.

**Amostragem.** Para um segmento maior que 16 elementos, são lidas cinco posições fixas. A amostra é ordenada localmente. O segundo valor define  $p_1$ , o terceiro serve como mediana de segurança e o quarto define  $p_2$ .

**Particionamento dual.** Três índices mantêm as zonas menor que  $p_1$ , entre  $p_1$  e  $p_2$  e maior que  $p_2$ . O elemento trazido da direita permanece desconhecido e é reavaliado.

**Tratamento de platôs.** Se  $p_1 = p_2$ , todos os elementos do corredor central são iguais ao limiar e não precisam de recursão.

**Fallback de progresso.** Se  $p_1$  e  $p_2$  são distintos, mas nenhum elemento sai do corredor central, uma partição ternária usa a mediana da amostra. O bloco igual ao pivô fica congelado e pelo menos uma chave deixa de participar das chamadas seguintes.

**Corte de segmentos pequenos.** Segmentos de tamanho no máximo 16 são ordenados por Insertion Sort.

## 4.3 Pseudocódigo

```
ACPS(A):
  B ← cópia de A
  se |B| ≤ 1: retorne B

  examine pares adjacentes de B
  se B é não decrescente: retorne B
  se B é não crescente:
    reverta B por trocas simétricas
    retorne B

ORDENAR(B, 0, |B|-1)
  retorne B

ORDENAR(B, low, high):
  enquanto low < high:
    se high-low+1 ≤ 16:
      INSERTION(B, low, high)
      retorne

  S ← valores de low, 1/4, 1/2, 3/4 e high
  ordene S
  p1 ← S[1]; mediana ← S[2]; p2 ← S[3]

  (L, R) ← PARTICAO_DUAL(B, low, high, p1, p2)

  se p1 ≠ p2 e L = low e R = high:
    (E1, E2) ← PARTICAO_TERNARIA(B, low, high, mediana)
    segmentos ← [low..E1-1, E2+1..high]
  senão:
    segmentos ← [low..L-1, R+1..high]
  se p1 ≠ p2:
    adicione L..R

  descarte segmentos vazios ou unitários
  aplique ORDENAR aos segmentos menores
  continue o enquanto usando o maior segmento
```

## 5 Exemplo numérico

Considere o vetor com 17 elementos:

[9, 1, 7, 3, 8, 2, 6, 4, 5, 0, 10, 2, 8, 1, 7, 3, 6]

A sondagem encontra uma descida em 9 para 1 e posteriormente uma subida em 1 para 7. Portanto, o vetor não é monotonicamente ordenado e segue para a fase recursiva.

As cinco posições amostradas contêm 9, 8, 5, 8 e 6. A amostra ordenada é [5, 6, 8, 8, 9]; logo,  $p_1 = 6$ , mediana = 8 e  $p_2 = 8$ .

Depois do particionamento, a ordem interna de cada zona não é especificada, mas as propriedades são:

| Zona     | Condição          | Multiconjunto de valores  |
|----------|-------------------|---------------------------|
| Inferior | $x < 6$           | 0, 1, 1, 2, 2, 3, 3, 4, 5 |
| Central  | $6 \leq x \leq 8$ | 6, 6, 7, 7, 8, 8          |
| Superior | $x > 8$           | 9, 10                     |

Como  $p_1$  e  $p_2$  são diferentes, as três zonas ainda podem exigir ordenação interna. A zona superior possui apenas dois elementos e usa Insertion Sort. As demais repetem o procedimento. Ao final, obtém-se:

[0, 1, 1, 2, 2, 3, 3, 4, 5, 6, 6, 7, 7, 8, 8, 9, 10]

## 6 Invariantes e prova de corretude

### 6.1 Sondagem inicial

Depois de processar os pares de índices 0 até i:

- `is_non_decreasing` e verdadeiro se, e somente se, nenhum par processado viola  $A[j] \leq A[j+1]$ ;
- `is_non_increasing` e verdadeiro se, e somente se, nenhum par processado viola  $A[j] \geq A[j+1]$ .

Os indicadores são verdadeiros antes do primeiro par. Cada novo par apenas elimina as hipóteses que contradiz. Se uma hipótese permanece ao final, todos os pares adjacentes satisfazem a relação correspondente. Uma sequência não decrescente já está ordenada. A reversão de uma sequência não crescente produz uma sequência não decrescente, inclusive quando há valores repetidos.

### 6.2 Particionamento dual

Antes de cada iteração do laço principal, valem:

- $A[\text{low}:\text{left}] < p_1$ ;
- $p_1 \leq A[\text{left}:\text{current}] \leq p_2$ ;
- $A[\text{right}+1:\text{high}+1] > p_2$ ;
- $A[\text{current}:\text{right}+1]$  ainda não foi classificado.

Inicialmente, as três primeiras regiões são vazias. Se o elemento corrente é menor que  $p_1$ , ele é trocado com `left` e ambos os limites avançam. Se é maior que  $p_2$ , ele é trocado com `right`; o limite direito recua e o valor trazido volta a ser avaliado. Caso contrário, o elemento pertence ao corredor central e apenas `current` avança. Em todos os casos, as regiões classificadas preservam suas propriedades.

A quantidade `right-current+1` diminui em cada iteração, pois `current` aumenta ou `right` diminui. Portanto, o laço termina. Quando `current > right`, a região desconhecida é vazia e todo o segmento satisfaz a partição desejada.

### 6.3 Particionamento ternário de segurança

O fallback mantém invariantes análogos: valores menores que o pivô ficam antes de `left`, iguais ficam entre `left` e `current`, maiores ficam depois de `right`, e a zona restante é desconhecida. Como a mediana pertence ao segmento, a zona de iguais não é vazia. Esse bloco é congelado e as chamadas seguintes recebem apenas segmentos estritamente menores.

### 6.4 Corretude global

A prova segue por indução no tamanho do segmento. Segmentos de tamanho zero ou um estão ordenados. Segmentos de tamanho até 16 são ordenados pelo Insertion Sort, cuja corretude é conhecida pelo invariante do prefixo ordenado.

Para um segmento maior, os particionamentos produzem zonas ordenadas entre si: qualquer valor da zona inferior é menor que os do corredor, e qualquer valor do corredor é menor ou igual aos da zona superior. Pela hipótese de indução, cada zona recursiva termina internamente ordenada. A concatenação das zonas é, portanto, ordenada. O congelamento é seguro porque uma zona entre limites iguais contém apenas valores iguais. O fallback garante progresso quando o particionamento dual não reduz o problema. Logo, ACPS termina e devolve uma permutação ordenada da entrada.

## 7 Análise assintótica

### 7.1 Custo das fases

A sondagem inicial custa no máximo duas comparações por par adjacente, portanto  $\Theta(N)$ . A amostra tem tamanho constante cinco; ordena-la custa  $O(1)$ . Cada particionamento examina e movimenta elementos dentro de um unico segmento, logo custa  $\Theta(m)$  para um segmento de tamanho  $m$ . O corte por Insertion Sort atua apenas em segmentos de tamanho limitado por 16, portanto seu custo por folha é constante em termos assintóticos globais.

### 7.2 Melhor caso

Uma entrada não decrescente exige exatamente  $N-1$  comparações  $<$  e nenhuma movimentação. Uma entrada estritamente decrescente exige duas comparações por par e  $N$  escritas quando  $N$  é par. Em ambos os casos:

$$T_{\text{best}}(N) = \Theta(N)$$

O mesmo limite vale para entradas com todos os elementos iguais, embora sejam realizadas duas avaliações relacionais por par.

### 7.3 Caso médio

Sob a hipótese de uma permutação aleatória de chaves distintas, as posições fixas constituem uma amostra aleatória sem reposição. O segundo e o quarto valores da amostra tendem a separar frações constantes da entrada. Uma recorrência balanceada idealizada é:

$$T(N) = 3 T(N/3) + cN$$

Pelo Teorema Mestre,  $T(N) = \Theta(N \log N)$ . Essa conclusão depende da distribuição: não é uma garantia para toda entrada, pois os pivôs são escolhidos deterministicamente a partir de posições fixas.

### 7.4 Pior caso

Uma entrada adversarial pode fazer uma partição isolar apenas uma quantidade constante de elementos. Nesse caso:

$$T(N) = T(N-c) + \Theta(N) = \Theta(N^2)$$

O fallback ternário evita ausência de progresso e remove o comportamento quadrático observado anteriormente em entradas binárias comuns, mas não transforma a amostragem determinística em uma garantia de balanceamento.

### 7.5 Espaço, in-place e estabilidade

O particionamento modifica a lista de trabalho sem vetor auxiliar proporcional ao segmento. Ao processar a maior partição por iteração e recorrer apenas nas menores, a profundidade da pilha é  $O(\log N)$ . Entretanto, a função pública cria `list(arr)` para preservar a entrada; por isso, a memória total observável é  $\Theta(N)$ .

O ACPS não é estável. Trocas distantes podem inverter a ordem relativa de objetos com a mesma chave. A suite inclui um contraexemplo automatizado. A implementação preserva a lista recebida e devolve uma nova lista ordenada.

## 8 Comparacao teorica

| Algoritmo             | Melhor                    | Medio                       | Pior               | Memoria auxiliar tipica | Estável |
|-----------------------|---------------------------|-----------------------------|--------------------|-------------------------|---------|
| Bubble com parada     | $\Theta(N)$               | $\Theta(N^2)$               | $\Theta(N^2)$      | $O(1)$                  | Sim     |
| Selection             | $\Theta(N^2)$             | $\Theta(N^2)$               | $\Theta(N^2)$      | $O(1)$                  | Nao     |
| Insertion             | $\Theta(N)$               | $\Theta(N^2)$               | $\Theta(N^2)$      | $O(1)$                  | Sim     |
| Merge                 | $\Theta(N \log N)$        | $\Theta(N \log N)$          | $\Theta(N \log N)$ | $\Theta(N)$             | Sim     |
| Quick mediana de três | $\Theta(N \log N)$        | $\Theta(N \log N)$          | $\Theta(N^2)$      | $O(\log N)$ esperado    | Nao     |
| DPES                  | $\Theta(N)$ em homogeneos | $\Theta(N \log N)$ esperado | $\Theta(N^2)$      | pilha recursiva         | Nao     |
| ACPS                  | $\Theta(N)$               | $\Theta(N \log N)$ esperado | $\Theta(N^2)$      | $O(\log N)$ no núcleo   | Nao     |

Merge Sort fornece a garantia temporal mais forte e estabilidade, mas requer memória linear. Quick Sort tende a realizar menos movimentações do que ACPS em dados aleatorios. DPES procura extremos em cada segmento, enquanto ACPS evita essa varredura dupla e usa uma amostra constante. ACPS se diferencia principalmente pela sondagem global e pelo tratamento explicito de estagnação e monotonicidade.

## 9 Metodologia experimental

Foram utilizadas as entradas  $N = 10, 100, 1.000$  e  $10.000$  nas distribuições:

- aleatória uniforme em  $[-10N, 10N]$ ;
- ordenada;
- reversa;
- repetida com cinco chaves possíveis;
- quase ordenada, com aproximadamente 5% de trocas aleatórias.

Cada combinação foi executada cinco vezes. Para cada (distribuição,  $N$ , repetição), o vetor foi gerado uma única vez e reutilizado por todos os algoritmos. A ordem dos algoritmos foi embaralhada deterministicamente em cada repetição. As sementes derivam da semente base 42 e foram armazenadas junto aos dados brutos.

O tempo foi medido com `time.perf_counter_ns`. O coletor de lixo foi desabilitado apenas durante cada medição e restaurado em seguida. Antes das medições, cada algoritmo executou um aquecimento sobre 256 elementos. Toda saída foi comparada integralmente com `sorted(data)`.

Os algoritmos quadráticos Bubble, Selection e Insertion foram limitados a  $N = 1.000$ . Essa decisao evita que o custo dos métodos deliberadamente quadráticos domine o experimento, sem remover a tendencia necessaria para a comparação. Os demais algoritmos foram executados até  $N = 10.000$ .

O ambiente registrado foi CPython 3.14.6 em Windows 11, arquitetura AMD64, processador Intel64 Family 6 Model 154, com Matplotlib 3.11.1. Tempos de parede dependem de carga do sistema, frequência do processador e ambiente; comparações e movimentações sao deterministicas para uma entrada fixa.

# 10 Resultados experimentais

## 10.1 Resumo em N igual a 10.000

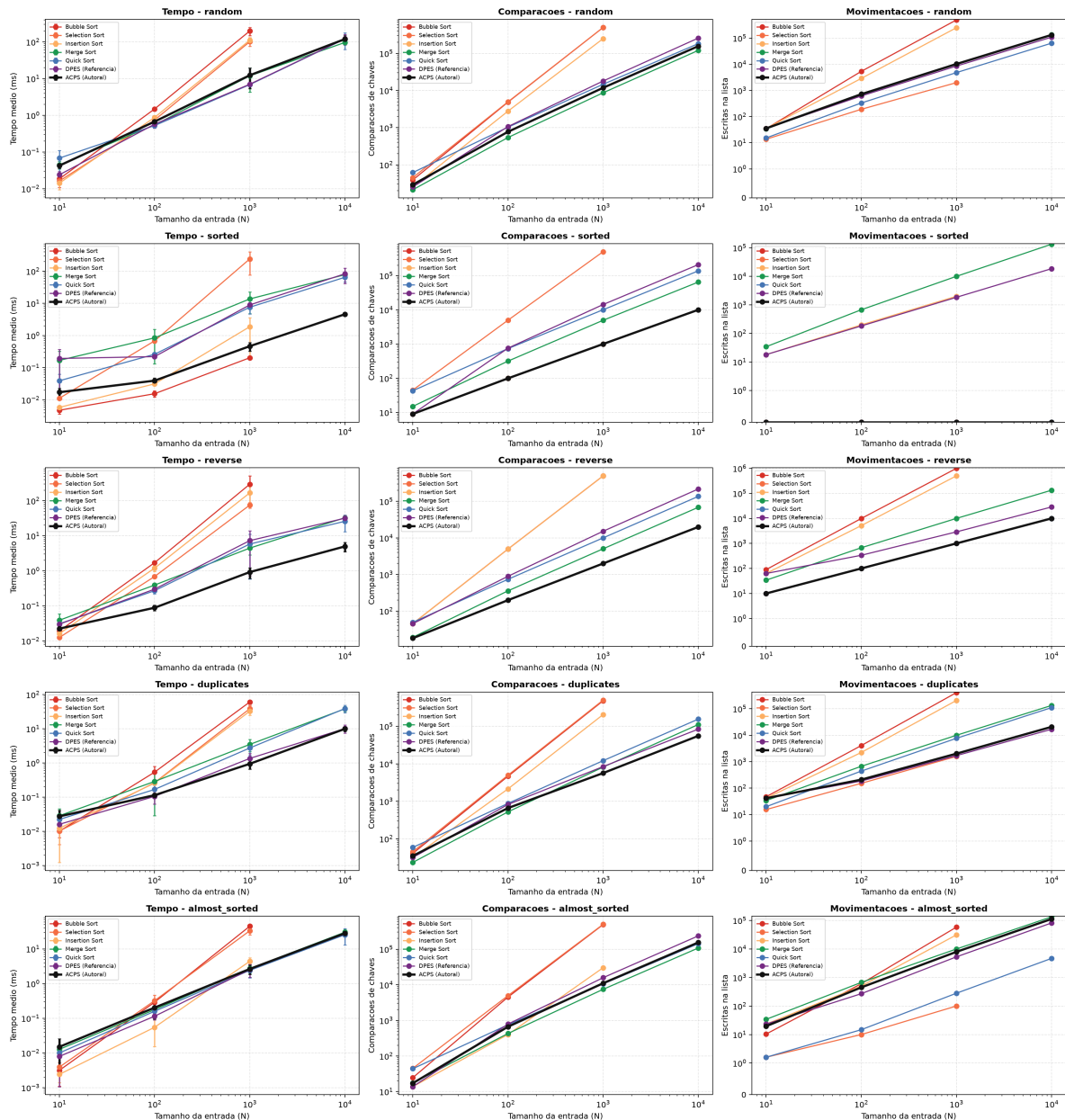
Os valores de tempo apresentam media e desvio padrão de cinco repetições. Os algoritmos quadráticos (Bubble, Selection, Insertion) foram omitidos desta tabela por ultrapassarem o limite de  $N = 1.000$  definido na metodologia; suas curvas aparecem nos gráficos da Seção 10.2. O Merge Sort e exibido no cenário aleatório; nos demais cenários foi executado normalmente e seus resultados completos estão em `data/benchmark_data.json`. A ausência do Merge nas linhas seguintes é intencional para destacar o comportamento diferenciado do ACPS frente aos cenários para os quais a sondagem de monotonicidade foi projetada.

| Cenário        | Algoritmo | Tempo ms           | Comparacoes | Movimentacoes |
|----------------|-----------|--------------------|-------------|---------------|
| Aleatório      | Merge     | 96,874 +/- 10,531  | 120.433     | 133.616       |
| Aleatório      | Quick     | 120,578 +/- 58,574 | 183.065     | 64.194        |
| Aleatório      | DPES      | 121,894 +/- 39,142 | 258.894     | 109.058       |
| Aleatório      | ACPS      | 119,748 +/- 29,059 | 156.558     | 133.011       |
| Ordenado       | Quick     | 64,742 +/- 18,250  | 137.247     | 0             |
| Ordenado       | DPES      | 83,079 +/- 41,463  | 208.805     | 18.542        |
| Ordenado       | ACPS      | 4,597 +/- 0,582    | 9.999       | 0             |
| Reverso        | Quick     | 25,930 +/- 12,926  | 137.263     | 10.000        |
| Reverso        | DPES      | 31,726 +/- 6,337   | 218.845     | 28.619        |
| Reverso        | ACPS      | 4,977 +/- 1,474    | 19.998      | 10.000        |
| Repetidos      | Quick     | 38,988 +/- 9,517   | 155.625     | 110.793       |
| Repetidos      | DPES      | 10,139 +/- 2,765   | 84.033      | 16.854        |
| Repetidos      | ACPS      | 9,942 +/- 1,592    | 55.573      | 20.793        |
| Quase ordenado | Quick     | 25,600 +/- 12,550  | 143.130     | 4.677         |
| Quase ordenado | DPES      | 28,154 +/- 3,464   | 236.731     | 81.756        |
| Quase ordenado | ACPS      | 28,322 +/- 1,123   | 154.893     | 112.687       |

O Merge Sort não possui otimização de melhor caso: seu número de comparações permanece próximo a  $N \log N$  independentemente da distribuição. Essa invariância é uma propriedade fundamental que o diferencia do ACPS — o qual reduz o trabalho a  $\Theta(N)$  em entradas monotonicamente ordenadas — e justifica a escolha de destacar os três algoritmos adaptativos (Quick, DPES, ACPS) nas entradas ordenada, reversa, repetida e quase ordenada.



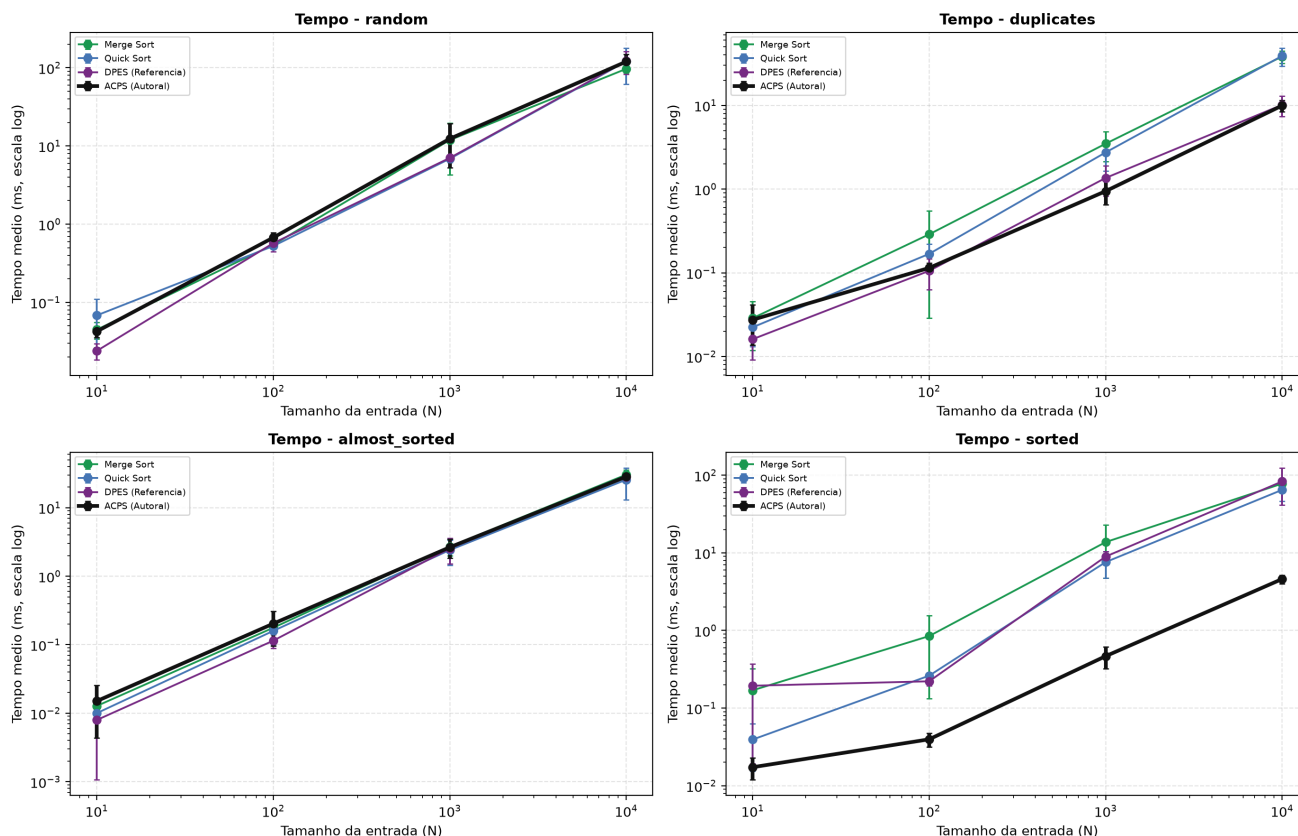
## 10.2 Visão geral



*Tempo, comparações e movimentações por cenário*

As escalas logarítmicas permitem visualizar no mesmo painel os métodos quadráticos e os métodos de divisão e conquista. As barras de erro representam um desvio padrão no tempo.

## 10.3 Algoritmos de maior escala

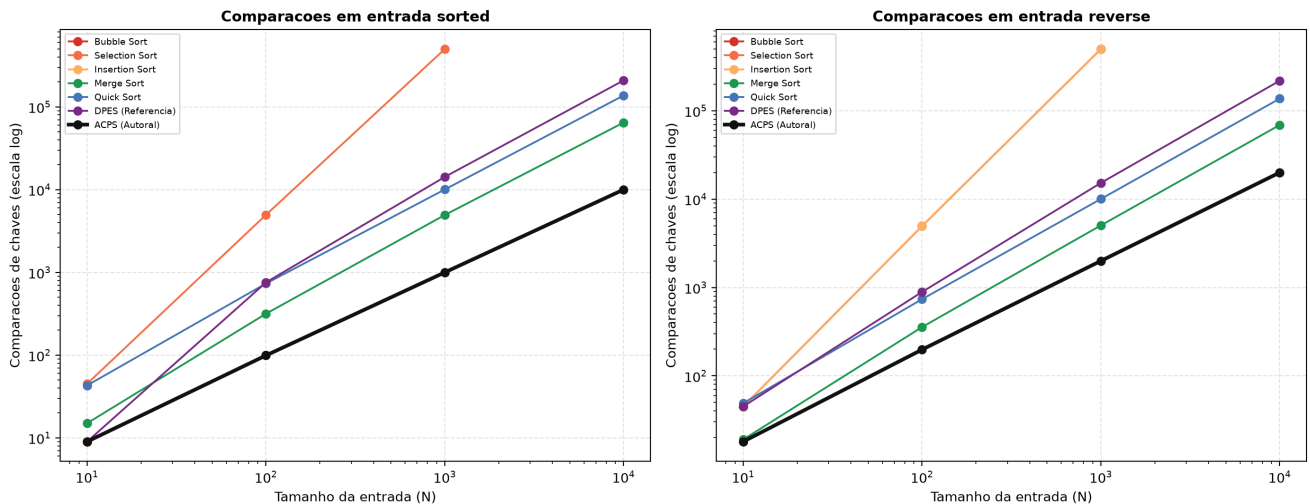


*Tempos dos algoritmos executados até N igual a 10000*

Em dados aleatorios, ACPS, Quick, Merge e DPES permaneceram na mesma ordem de grandeza temporal. Merge apresentou a menor media nessa execução. ACPS realizou menos comparações do que Quick e DPES, mas mais movimentações do que Quick; isso evidência que comparações isoladas não explicam integralmente o tempo.

Em entradas ordenadas e reversas, a sondagem produziu a vantagem esperada. Em entradas com cinco chaves, ACPS e DPES foram substancialmente mais rapidos do que Merge e Quick nas implementacoes avaliadas. Em entradas quase ordenadas, a sondagem termina cedo ao detectar as duas direcoes, de modo que ACPS não explora ordenação parcial alem do caso monotonicamente global.

## 10.4 Comparacoes em entradas monotonicamente ordenadas



Comparacoes em entradas ordenadas e reversas

Para entrada ordenada, ACPS executa  $N-1$  comparações. Para entrada reversa estrita, cada par exige  $< \text{falso} >$  e  $> \text{verdadeiro}$ , totalizando  $2(N-1)$ . Essa contagem substitui a versao anterior que contabilizava cada par reverso como uma única comparação.

## 11 Validacao da implementaçao

A suite final contem 22 testes. Os grupos cobrem:

- **Casos limite obrigatórios do enunciado:**  $N = 0$  (vetor vazio, teste 01) e  $N = 1$  (elemento único, teste 02), ambos verificando resultado correto, zero comparações e zero movimentações; e dois elementos (testes 03 e 04);
- entradas ordenadas e reversas;
- todos iguais, poucos valores e blocos repetidos;
- quase ordenados;
- inteiros negativos e pontos flutuantes;
- escalabilidade aleatória até  $N = 10.000$ ;
- entradas não crescentes com repetições;
- regressao binaria contra fallback quadrático;
- enumeracao exaustiva de todos os vetores de tamanho 0 a 7 sobre  $\{0, 1, 2\}$ ;
- preservação da entrada;
- igualdade entre o contador reportado e as chamadas relacionais observadas;
- contraexemplo de estabilidade;
- corretude dos seis algoritmos de comparação.

Todos os 22 testes passaram. A enumeracao exaustiva representa 3.280 vetores. O benchmark adicionou 625 execuções validadas, cada uma comparada com a ordenação da biblioteca Python.

## 12 Limitações e ameaças a validade

O ACPS não oferece pior caso  $O(N \log N)$ . Entradas adversariais podem explorar as cinco posições fixas e produzir partições desbalanceadas. Uma extensão possível seria randomizar a amostra ou adotar limite de profundidade com fallback para Heap Sort.

A adaptatividade global reconhece somente sequências inteiramente não decrescentes ou não crescentes. Ela não identifica múltiplas subsequências naturais como Timsort. O custo da sondagem é pequeno em média porque ela termina assim que encontra uma subida e uma descida, mas permanece uma passagem adicional em entradas monotonicamente ordenadas.

O estudo de tempo usa cinco repetições em uma única máquina e não constitui avaliação multiplataforma. Diferenças pequenas devem ser interpretadas com cautela quando os desvios padrão se sobrepõem. Os contadores dependem da convenção declarada e não representam diretamente instruções de máquina, acessos a cache ou consumo de energia.

A interface copia a entrada. Portanto, o núcleo é in-place, mas a função completa não atende a definição estrita de ordenar o objeto recebido sem memória linear adicional. Essa escolha preserva a compatibilidade com os códigos de apoio e evita efeitos colaterais durante os experimentos.

## 13 Declaração de autoria e uso de inteligência artificial

O ACPS é apresentado como adaptação estrutural autoral. Seus autores conceberam a combinação de sondagem de monotonicidade, amostragem posicional, corredor central, congelamento de platôs e políticas de fallback. Componentes conhecidos foram identificados e referenciados; não se reivindica autoria sobre Quick Sort, Dual-Pivot Quicksort, particionamento ternário ou Insertion Sort.

Foram utilizadas as seguintes ferramentas de inteligência artificial:

**Google Antigravity com Gemini.** Foi utilizado na fase inicial para auxiliar a automação de testes, a geração de gráficos com Matplotlib e a formatação de tabelas e arquivos.

**OpenAI Codex.** Foi utilizado para auditar a implementação e o enunciado, localizar inconsistências na contagem de comparações, detectar uma falha da baseline Quick Sort com duplicatas, construir testes exaustivos e de propriedades, revisar o caso degenerado com poucas chaves, reformular o benchmark reproduzível, organizar a análise assintótica e estruturar este relatório.

As principais modificações decorrentes dessa assistência foram: substituição do fallback de Insertion Sort sobre segmentos grandes por partição ternária; instrumentação explícita dos operadores relacionais; eliminação da chamada recursiva da maior partição; correção da baseline Quick Sort; reutilização dos mesmos datasets por todos os algoritmos; armazenamento das repetições e do desvio padrão; e ampliação da suíte para 22 testes.

Os resultados foram validados por testes unitários, enumeração exaustiva, comparação com `sorted(data)` em cada execução do benchmark e inspeção dos dados e gráficos regenerados. Os autores assumem responsabilidade pela revisão final, pelo domínio da implementação e pela defesa das escolhas e limitações descritas.

## 14 Reprodutibilidade

Na raiz do projeto:

```
python -m pip install -r requirements.txt
python -m unittest discover -s tests -v
python src/student_template.py
python benchmarks/benchmark.py --trials 5 --sizes 10,100,1000,10000
python scripts/build_report_pdf.py
```

O benchmark gera `data/benchmark_data.json`, `data/benchmark_summary.md` e os três arquivos PNG em `assets`. O JSON inclui metadados do ambiente, sementes e observações individuais. O script do PDF lê este Markdown e incorpora os gráficos produzidos.

## 15 Conclusao

O ACPS atingiu o objetivo academico de combinar projeto, formalizacao, implementação e avaliação crítica. A sondagem monotônica oferece melhor caso  $\Theta(N)$  e comportamento especialmente favoravel em entradas ordenadas e reversas. A amostragem e o particionamento dual mantêm desempenho esperado  $\Theta(N \log N)$  em permutacoes aleatórias, enquanto o fallback ternário evita estagnação em conjuntos com poucas chaves.

Os experimentos não demonstram superioridade universal. Merge Sort apresentou menor tempo médio em dados aleatorios na maior entrada; Quick Sort realizou menos movimentações; e o ACPS conserva pior caso  $\Theta(N^2)$ . A principal conclusão e, portanto, condicional: o mecanismo autoral e correto, reproduzível e eficiente nas distribuições para as quais foi projetado, mas suas vantagens dependem da estrutura da entrada.

## Referências

1. AUMULLER, Martin; DIETZFELBINGER, Martin. *Optimal Partitioning for Dual-Pivot Quicksort*. 2013. Disponível em: <https://arxiv.org/abs/1303.5217>.
2. NEBEL, Markus E.; WILD, Sebastian. *Pivot Sampling in Dual-Pivot Quicksort*. 2014. Disponível em: <https://arxiv.org/abs/1403.6602>.
3. SEDGEWICK, Robert; WAYNE, Kevin. *Algorithms, 4th Edition: Quicksort*. Princeton University. Disponível em: <https://algs4.cs.princeton.edu/23quicksort/>.
4. UNIPAMPA. *Trabalho Prático 1 - Metodos de Ordenacao Autorais*. Enunciado e pacote de códigos disponibilizados na disciplina de Análise e Projeto de Algoritmos, 2026.
5. YAROSLAVSKIY, Vladimir. *Dual-Pivot Quicksort*. Algoritmo incorporado e analisado em implementacoes modernas de ordenação e nos trabalhos citados acima.